

Beginner's Guide to GitLab CI/CD Pipelines

Learning GitLab CI/CD helps you stop deploying code manually and start automating your workflow like a pro. This guide breaks down the core pieces of a single .gitlab-ci.yml file so you can build your first pipeline with confidence.

What you'll learn:

1. Defining Your Pipeline Stages
2. Running the Build Job
3. Automating Tests
4. Deploying to Production

Aman Raj Singh

Cloud Operations Engineer @ iManage

[linkedin.com/in/mramanrajsingh](https://www.linkedin.com/in/mramanrajsingh)

1

TIP 1 OF 4

1. Defining Your Pipeline Stages

Stages are the chronological checkpoints of your automation. By default, GitLab runs builds, tests, and deploys in order. If the build fails, the pipeline stops immediately so you don't waste time testing broken code. Think of it like a factory assembly line where each station must pass inspection.

```
$stages:  
  - build  
  - test  
  - deploy
```

Cloud & DevOps Daily

GitLab CI/CD: Build, Test & Deploy With One .gitlab-ci.yml

Sunday, October 04, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Running the Build Job

The build job prepares your application for action. It usually downloads packages, compiles code, or bundles frontend assets. You define which Docker container image to use so your environment is always clean and consistent. This guarantees your code runs the same way on GitLab as it does on your computer.

```
$build_code:
  stage: build
  image: node:18
  script:
    - npm install
    - npm run build
```

Cloud & DevOps Daily

GitLab CI/CD: Build, Test & Deploy With One .gitlab-ci.yml

Sunday, October 04, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Automating Tests

Once your app is built, automated tests run to catch silly mistakes before they reach production. If a test fails, the pipeline turns red and blocks deployment. This saves you from those stressful 3 AM moments when a simple typo breaks the live website.

```
$run_tests:
  stage: test
  image: python:3.9
  script:
    - pip install -r requirements.txt
    - pytest
```

Cloud & DevOps Daily

GitLab CI/CD: Build, Test & Deploy With One .gitlab-ci.yml

Sunday, October 04, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Deploying to Production

The final step takes your polished, tested application files and sends them to your server. You can use SSH, cloud CLIs, or deployment scripts here. This ensures every single release follows the exact same reliable steps without human error.

```
$deploy_app:
  stage: deploy
  script:
    - echo "Deploying application to production server..."
    - ./deploy.sh
```

Cloud & DevOps Daily

GitLab CI/CD: Build, Test & Deploy With One .gitlab-ci.yml

Sunday, October 04, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Keep your .gitlab-ci.yml file in the root folder of your repository.
- + Use specific Docker image versions like node:18 instead of node:latest.
- + Store sensitive API keys and passwords in GitLab CI/CD Secret Variables.
- + Test small changes locally or use a separate branch before merging to main.
- + Keep your build scripts simple and easy for your team to read.

Watch Out For

- ! Hardcoding passwords and secret tokens directly inside the YAML file.
- ! Skipping the test stage and deploying straight to production.
- ! Forgetting to define stages before assigning them to individual jobs.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh